BugCapsule: Deterministic Runtime Bug Capsules for Reducing Agent Context and Improving Debugging Accuracy

Arnav Roy Choidorj Bayarkhuu

Abstract

Large language model coding agents are often asked to debug failures in repositories that contain substantially more code than is relevant to the failing behavior. BugCapsule is a local-first Model Context Protocol (MCP) tool that converts an observed TypeScript failure into a small executable repository slice called a capsule. The capsule preserves the failing reproduction, source files on the error path, generated metadata, and a strict apply-back workflow. This report summarizes the BugCapsule implementation and evaluates both its context-reduction behavior and its debugging-accuracy motivation using ten purpose-built stress-test projects plus one demo storefront repository. Across the ten stress tests, BugCapsule reduced deterministic input context from 193,254 tokens to 44,375 tokens, a 77.0% aggregate reduction. In the demo storefront, SWE-1.6 Fast, the exact Cognition/Windsurf coding model tier used in the evaluation environment, failed to fix the checkout issue in the full-repository setting, while the same model using BugCapsule fixed the issue. This result illustrates that BugCapsule is not only a token-reduction mechanism; it can also improve debugging outcomes by replacing broad repository search with a smaller executable failure context.

1 Introduction

Debugging with AI coding agents has a context-selection problem. A user-visible failure may be caused by a small group of functions, but the agent can be exposed to the entire repository, including unrelated modules, UI assets, build outputs, and support code. This wastes input tokens and increases the chance that the model reasons over irrelevant files, proposes symptom patches, or misses the narrow data path that produced the error.

SWE-1.6 Fast is the coding model tier used for the reported evaluation runs. SWE-1.6 is a software-engineering agent model from Cognition, made generally available in Windsurf; Cognition describes it as optimized for software-engineering agents and model user experience, with a fast tier advertised at up to 950 tokens per second.[1] Throughout this report, *SWE-1.6 Fast* refers specifically to that fast-tier model operating either directly on the repository or through BugCapsule's MCP workflow.

BugCapsule addresses this by creating an executable debugging capsule. A capsule is not merely a static file list. It contains a failing command, the source slice required to reproduce the error, generated reproduction files when needed, metadata that maps capsule files back to the original repository, and workflow gates that force the agent to inspect, reproduce, verify, and apply only a verified fix. The intended effect is therefore twofold: reduce the input context and increase the probability that an agent debugs the true failure rather than navigating the full repository opportunistically.

The demo storefront provides a concrete motivation. In that repository, SWE-1.6 Fast failed to repair the checkout bug when operating against the normal repository context. With BugCapsule, the same model was given a generated failing reproduction, a mapped capsule file set, and a strict verify-before-apply workflow; under those conditions it fixed the issue. This is a binary functional outcome, not a broad statistical accuracy benchmark by itself. It matters because it demonstrates the mechanism by which context reduction can affect correctness: the agent is constrained to a smaller, executable failure instance and must validate the fix before apply-back.

The quantitative evaluation focus of this report is practical: whether BugCapsule lowers the context payload that an agent needs for debugging while preserving an executable failure. The stress-test suite was deliberately designed with substantial unrelated code so that a successful slicer should remove large amounts of context while preserving the one real runtime bug in each project.

2 System Overview

BugCapsule is implemented as a TypeScript monorepo with two main packages:

- `@bugcapsule/core`: capsule creation, failure capture, runtime probing, import-graph slicing, mock generation, verification, and apply-back logic.
- `@bugcapsule/mcp`: a stdio MCP server exposing tools such as `bugcapsule_create_from_runtime`, `bugcapsule_create_from_command`, `bugcapsule_fix_step`, `bugcapsule_verify`, and `bugcapsule_apply_patch`.

The tech stack is Node.js, TypeScript, npm workspaces, ESM modules, Vitest, and the MCP TypeScript SDK. The core package uses `fast-glob` and `minimatch` for file discovery and exclusion, `ts-morph` for TypeScript

[1] `https://cognition.ai/blog/swe-1-6`

import graph construction, and `diff` for patch generation. Evaluation reports use deterministic token counting through `js-tiktoken`; the reports analyzed here used the `o200k_base` tokenizer and the "SWE-1.6 Fast" pricing profile.

The MCP design is central to the accuracy argument. Rather than asking the model to manually decide which files matter, BugCapsule exposes deterministic tools that discover the failure, construct the capsule, run the reproduction, and enforce apply-back. The model still writes the fix, but the surrounding workflow constrains the debugging task to a smaller, executable, and verified problem instance.

3 Capsule Creation

BugCapsule supports two main creation paths: command-based failures and runtime URL failures. Both paths converge on the same capsule builder.

3.1 Command-Based Capsules

For a known failing command, BugCapsule runs the command in the original repository, captures stdout and stderr under `.bugcapsule/captures`, extracts a failure summary, and parses stack frames relative to the repository root. If the command passes, no useful failing capsule is created. If the command fails, the parsed failure becomes the root for slicing.

3.2 Runtime Capsules

For a local URL plus a broad symptom, BugCapsule probes the page without requiring the user to know a failing command. The runtime path performs the following steps:

1. Fetch the target page and extract same-origin interactions, especially `fetch(...)` calls embedded in the page.
2. Score interactions against symptom tokens and issue HTTP requests to likely endpoints.
3. Treat HTTP error responses or structured error payloads as candidate failures.
4. Extract the error message and stack trace when present.
5. Attempt to discover sample input payloads and select a narrow exported function that reproduces the error.
6. If function-level reproduction is unavailable, fall back to a server-interaction repro that imports the server module, binds an isolated port, and replays the failing route.

Generated runtime repros are written under `.bugcapsule/repros` and then included in the capsule as non-editable support files. For richer runtime captures, BugCapsule can generate lineage instrumentation that wraps exported functions and records request-boundary inputs as data flows through source modules. This helps preserve causality when the failure appears downstream from the user-visible interaction.

3.3 Slicing and Packaging

The slicer chooses roots from stack frames, inferred test files, command tokens, and optional include globs. It builds a TypeScript import graph and traverses local imports up to configured file and depth limits. It also searches for suspected upstream causes from the stack trace so that input transformation code can be included even when the throw site is downstream. Secret-like paths and configured exclusions such as `node_modules`, `dist`, coverage, and framework build directories are ignored.

The resulting capsule contains copied source and test files, generated mocks for selected external modules, a capsule-specific `package.json`, a capsule-specific `tsconfig.json`, a `capsule.json` manifest, and a README. The manifest records hashes, file mappings, the original failure, the capsule run command, and the original-to-capsule file relationship.

3.4 Strict Fix Workflow

BugCapsule uses a deterministic workflow enforced by `bugcapsule_fix_step`. The intended order is:

`inspect` → `reproduce_initial` → `verify_capsule` → `apply_patch`

Apply-back is intentionally constrained. The tool maps modified editable capsule files back to original paths, requires the verified capsule file set to match the apply attempt, can require a clean original worktree, and writes patch artifacts under `.bugcapsule/patches`. This avoids applying an unverified or unrelated edit to the source repository.

3.5 Determinism in the MCP Workflow

MCP tools are often invoked by nondeterministic agents: two runs of the same model may choose different tool orders, inspect different files, or apply a patch after incomplete verification. BugCapsule tries to make the tool layer deterministic even when the agent is not. It does this by turning the debugging process into an explicit state machine rather than a set of optional helper calls.

Each strict capsule stores workflow state in `.bugcapsule/workflows/<capsule-id>.json`. The state machine exposes exactly one required next action at each step: `inspect`, then `reproduce_initial`, then `verify_capsule`, then `apply_patch`. Calls outside the required order are rejected and return the next required tool call. This makes the MCP interaction reproducible at the protocol level: the agent can write different fixes,

but it cannot skip the failure reproduction or apply an unverified patch through the strict workflow.

BugCapsule also records receipts for deterministic auditability. Command receipts include the command, working directory, exit code, stdout and stderr paths, output hashes, duration, locked-file integrity status, and a hash of the editable capsule file set. Non-editable files such as `capsule.json`, generated repros, and package/config files are treated as locked and checked for modification. Apply-back is permitted only after a passing `verify_capsule` receipt, and only if the current editable-file-set hash exactly matches the hash that passed verification. If the agent changes a capsule file after verification, BugCapsule resets the workflow to require verification again.

This design separates deterministic orchestration from model creativity. The model can still reason and patch source files, but the MCP server owns failure capture, file mapping, verification receipts, integrity checks, and apply gating. In workplace settings, this is important because the output is not just a suggested diff; it is a diff tied to a replayable failure and a recorded verification sequence.

4 Evaluation Design

The evaluation used eleven HTML reports: ten from `Desktop/BugCapsuleStressTesting` and one from `Desktop/BugCapsuleDemo`. Each report compares a deterministic full-repository context payload against the generated capsule payload. The reports exclude generated build artifacts, `node_modules`, lockfiles, and binaries by default. Therefore the measured value is not wall-clock agent cost; it is a reproducible quantitative context-size comparison under the same inclusion rules.

The ten stress projects each expose one runtime bug at `http://localhost:4177/`. They were designed to be error-producing, fixable by BugCapsule, and surrounded by unrelated but compilable TypeScript backoffice subsystems. The unrelated subsystem is important because it tests whether BugCapsule removes real source files rather than only trivially small repositories. These stress-test results are quantitative: tokens, files, storage, and percentage reduction are measured directly from the generated evaluation reports. The demo adds a binary functional comparison: SWE-1.6 Fast failed without BugCapsule and passed with BugCapsule.

5 Results

Table 1 shows the per-case stress-test results. The reduction is computed as

$$\text{Reduction} = 1 - \frac{\text{capsule tokens}}{\text{full repo tokens}}.$$

The stress suite produced consistent reductions. Aggregate full-repository context was 193,254 tokens; aggregate capsule context was 44,375 tokens; aggregate tokens removed were 148,879. The mean per-case reduction was 77.0% with a standard deviation of 1.7 percentage points. The minimum reduction was 74.2% and the maximum was 79.0%.

Table 2 includes the demo storefront. The demo still reduced context, but less dramatically than the stress suite. Its full context was 18,227 tokens and its capsule was 13,793 tokens. The report shows that the generated runtime repro and capsule metadata were large contributors: the demo capsule's generated repro was 4,342 tokens and `capsule.json` was 4,271 tokens. In contrast, the stress fixtures had a deliberately larger disconnected subsystem that could be removed while retaining a compact failing route.

5.1 Demo Accuracy Outcome

The demo storefront is the most important functional outcome. SWE-1.6 Fast failed to fix the checkout issue when used directly on the demo repository, but SWE-1.6 Fast plus BugCapsule fixed the issue. The capsule did not merely shorten the input by 24.3%; it changed the debugging setup. The model received the failing reproduction as an executable command, a focused set of mapped checkout files, runtime lineage and manifest context, and a workflow that required the capsule to pass before applying the patch back to the original project.

This case supports the central hypothesis behind BugCapsule: reduced context can improve accuracy when the removed context is mostly distractor code and the retained context is executable, failure-specific, and tied to verification. Even when token reduction is modest, as in the demo, the structure of the capsule can still improve the agent's ability to produce a correct fix.

6 Discussion

The stress-test results show BugCapsule behaving as intended when the repository contains a narrow failing runtime path plus unrelated code. In those conditions, it reliably removes about three quarters of the deterministic context while preserving an executable reproduction. This is the target use case: an agent receives less code, but still gets a failing command and enough mapped source to fix and verify the bug. The debugging benefit is not simply lower cost; it is a higher signal-to-noise ratio around the failure.

The demo result is also useful because it reveals the lower bound imposed by capsule overhead. A capsule is not free: it contains a manifest, README, generated repro, package metadata, and sometimes lineage logic. When the original repository is small or the generated runtime repro is large, overhead can consume much of the savings. This is why the demo achieves 24.3% rather than the stress suite's 77.0%. The result does not indicate failure; it indicates that context reduction depends on the ratio of irrelevant repository code to required reproduction scaffolding. More

Table 1: Stress-test evaluation results. Each case contains one runtime bug and a larger unrelated TypeScript subsystem.

Case	Bug class	Full tokens	Capsule tokens	Removed	Reduction	Files
01	Checkout digital delivery	19,487	5,030	14,457	74.2%	27 → 11
02	Support SLA routing	19,327	4,658	14,669	75.9%	27 → 11
03	Subscription proration	19,369	4,242	15,127	78.1%	27 → 9
04	Catalog search parser	19,150	4,225	14,925	77.9%	27 → 10
05	Inventory bin reservation	19,369	4,356	15,013	77.5%	27 → 10
06	Calendar reminder time zones	19,403	4,078	15,325	79.0%	27 → 9
07	Payment webhook signature	19,290	4,377	14,913	77.3%	27 → 10
08	CSV import decoder	19,092	4,130	14,962	78.4%	27 → 10
09	Report rollup cache	19,388	4,297	15,091	77.8%	27 → 10
10	Shipping label address	19,379	4,982	14,397	74.3%	27 → 15

Table 2: Aggregate context reduction summary.

Dataset	Full tokens	Capsule tokens	Reduction
Stress suite total	193,254	44,375	77.0%
Stress suite mean	19,325	4,438	77.0%
Demo storefront	18,227	13,793	24.3%

importantly, the demo shows that token reduction is an incomplete proxy for usefulness: BugCapsule enabled SWE-1.6 Fast to fix a bug it failed to fix without the capsule.

The stress cases also validate the newer runtime fallback behavior. The projects expose broad URL symptoms such as a button failing on click; they do not provide explicit reproduction scripts to the agent. BugCapsule still finds the failing same-origin endpoint, generates a hidden runtime repro, and creates a failing capsule. This matters for user workflows because many front-end or local-app bugs are first observed as UI symptoms rather than known test commands.

7 Threats to Validity

The evaluation measures deterministic context payloads, not complete end-to-end agent token usage. Actual agent behavior may read fewer files in a no-capsule run or may perform additional reasoning and verification steps in a capsule run. The evaluation therefore supports the claim that BugCapsule reduces available debugging context, not that every real agent session will realize exactly the same billing reduction. The SWE-1.6 Fast demo outcome is a single binary pass/fail comparison rather than a statistically powered accuracy benchmark; a broader agent-accuracy evaluation would require repeated no-capsule and capsule runs over many repositories and models.

The stress projects were synthetic. They were intentionally designed to contain one real bug plus unrelated backoffice code, so they are favorable to any slice-based approach. This is appropriate for stress testing context reduction, but broader evaluation should include mature real-world repositories with more tangled dependency graphs, heavier framework conventions, and failures that cross process or service boundaries.

Finally, the reports use `o200k_base` as a deterministic tokenizer proxy. For non-OpenAI providers or IDE-hosted models, provider billing tokenization may differ. The relative reductions should remain meaningful because both baseline and capsule are measured by the same tokenizer, but exact provider bills require provider usage logs.

8 Conclusion

BugCapsule converts runtime or command failures into executable, mapped, and verified debugging capsules. Its implementation combines MCP tools, runtime probing, stack parsing, generated repros, TypeScript import-graph slicing, mock generation, and strict apply-back workflow gates. In the ten-project stress suite, this design reduced deterministic debugging context by 77.0% overall while keeping each bug reproducible. The demo storefront result, 24.3%, shows that capsule overhead is visible in smaller or more coupled repositories, but the same demo also shows the accuracy motivation: SWE-1.6 Fast failed without BugCapsule and succeeded with BugCapsule.

The strongest workplace argument for BugCapsule is in large codebases, where debugging context is noisy, ownership boundaries are complex, and agents can easily spend attention on irrelevant modules. Enterprise repositories often contain multiple product surfaces, generated code, legacy utilities, feature flags, integrations, and infrastructure glue around a small failing path. BugCapsule's value grows in that environment because every unrelated subsystem excluded from the capsule reduces distraction while the strict workflow still preserves the parts that matter: an executable reproduction, mapped editable files, locked metadata, verification receipts, and controlled apply-back. The result is a debugging unit that can be handed to an agent with less context risk and more process discipline than a broad repository prompt.

For workplace engineering teams, this suggests a practical deployment model. Developers can report a local URL symptom or failing command; BugCapsule can convert it

into a reproducible capsule; an agent can fix the capsule; and the MCP workflow can apply only the verified mapped changes. That pipeline is valuable not only because it saves tokens, but because it makes agent debugging more reviewable, repeatable, and auditable. In large repositories, those properties are often more important than the raw token savings: they reduce false fixes, make patches easier to inspect, and give teams a deterministic record of what failed, what was changed, and what passed before the fix touched the original codebase.